

Inteligencia Artificial Generativa Avanzada y Sistemas Multi-Agente

Apuntes de clase — UTDT
Prof. Francisco Traversaro

Índice

1. De vectores a un LLM — Fundamentos matemáticos	3
1.1. El camino hacia un LLM	3
1.2. Vectores, producto interno y embeddings	3
1.3. Aplicaciones lineales y matrices	3
1.4. Unidad afín + no linealidad	3
1.5. Aprendizaje supervisado como optimización	4
1.6. Gradiente y descenso por gradiente	4
1.7. Backpropagation como diferenciación automática	4
1.8. Notebook: Motor de autograd (micrograd)	4
2. Attention is All You Need — Transformers	5
2.1. El punto de partida	5
2.2. Del texto a tokens	5
2.3. El bigrama: el modelo más simple	5
2.4. Promediar el pasado: darle memoria	5
2.5. Self-Attention	6
2.6. Multi-Head Attention	6
2.7. El bloque Transformer completo	6
2.8. Encoder vs Decoder	7
3. Detrás de la IA — Tokenizers	7
3.1. Por qué importa la tokenización	7
3.2. De texto a bytes	7
3.3. Byte-Pair Encoding (BPE)	7
3.4. Encode y Decode	8
3.5. Regex splitting (GPT-2/GPT-4)	8
3.6. Tokens especiales	8
3.7. Tokenizers reales y algoritmos	8
3.8. Notebook: Tokenización	9
3.9. Notebook: “¿Qué es realmente la IA?”	9
4. Embeddings	10
4.1. Parte 1: Word Embeddings	10
4.1.1. Los embeddings de un LLM se crean dentro del modelo	10
4.1.2. Similitud coseno	10
4.1.3. One-hot: identidad, cero significado	10

4.1.4.	Hipótesis distribucional y conteo	10
4.1.5.	Word2Vec: aprender prediciendo	10
4.1.6.	GloVe: contar y después aprender	11
4.1.7.	Techo: static embeddings y polisemia	11
4.1.8.	Sesgos	11
4.2.	Parte 2: Sentence Embeddings	11
4.2.1.	El problema	11
4.2.2.	Baseline: promediar word vectors	11
4.2.3.	BERT: el encoder	12
4.2.4.	SBERT (Sentence-BERT)	12
4.2.5.	Embedders basados en decoder (GPT)	12
4.2.6.	Tres “embeddings” distintos	12
4.2.7.	Búsqueda semántica = la “R” de RAG	12
4.2.8.	Los tres modelos de un RAG	13
4.2.9.	Qué embedder usar (2026)	13
4.2.10.	Lo que falta para RAG en serio (Semana 8)	13
4.3.	Notebook: Word Embeddings (N1)	13
4.4.	Notebook: Sentence Embeddings (N2)	13

Lecturas recomendadas	15
------------------------------	-----------

Clase 1: De vectores a un LLM — Fundamentos matemáticos

1.1 El camino hacia un LLM

Todo LLM se apoya en una cadena de bloques fundamentales:

Vectores/embeddings $\rightarrow$ **Red neuronal** $\rightarrow$ **Atención** $\rightarrow$ **Transformer** $\rightarrow$ **LLM**

Los símbolos clave de la clase: $\mathbb{R}^n$ (espacios vectoriales), $\mathbf{w}^\top \mathbf{x}$ (producto interno), $W\mathbf{x} + \mathbf{b}$ (capa afín), ϕ (activación), $L(\theta)$ (pérdida), ∇L (gradiente), AD (diferenciación automática).

1.2 Vectores, producto interno y embeddings

- Un vector $\mathbf{x} = (x_1, \dots, x_n) \in \mathbb{R}^n$ tiene dos lecturas: n -tupla de coordenadas (álgebra) y punto/dirección con norma (geometría).
- En deep learning, todo dato (token, imagen) se codifica como vector en dimensión alta: **representación distribuida**.
- **Embeddings**: vectores aprendidos por palabra. La geometría del espacio captura significado:

$$\text{vec}(\text{rey}) - \text{vec}(\text{hombre}) + \text{vec}(\text{mujer}) \approx \text{vec}(\text{reina})$$

- **Producto interno**: $\mathbf{w}^\top \mathbf{x} = \sum_i w_i x_i = \|\mathbf{w}\| \|\mathbf{x}\| \cos \theta$. Mide alineación/proyección entre vectores. Base de la **similitud coseno**.
- En atención, el peso de un token sobre otro es el producto interno de sus vectores query y key (escalado por $\sqrt{d}$, normalizado con softmax).

1.3 Aplicaciones lineales y matrices

- $W \in \mathbb{R}^{m \times n}$ representa $\mathbb{R}^n \rightarrow \mathbb{R}^m$. Cada componente de la salida es un producto interno de una fila con la entrada.
- Una **capa lineal** son m productos internos en paralelo.
- Componer capas lineales = multiplicar matrices. El producto matriz-matriz domina el costo del entrenamiento $\Rightarrow$ hardware paralelo (GPU).
- **SVD** ($W = U\Sigma V^\top$): toda aplicación lineal es rotación $\cdot$ escala $\cdot$ rotación. Relevante para LoRA y atención de rango bajo.

1.4 Unidad afín + no linealidad

- Unidad: $a = \phi(\mathbf{w}^\top \mathbf{x} + b)$. Capa: $\mathbf{h} = \phi(W\mathbf{x} + \mathbf{b})$.
- **Sin ϕ , apilar capas no agrega expresividad**: la composición de afines es afín. La no linealidad rompe eso.
- Activaciones comunes: ReLU, sigmoide (σ), tanh.
- **Teorema de aproximación universal**: una red con una capa oculta y no linealidad no polinomial aproxima cualquier función continua sobre un compacto (Cybenko 1989, Hornik 1991). No dice cuántas neuronas ni si el entrenamiento las encuentra.

1.5 Aprendizaje supervisado como optimización

- Datos (x_i, y_i) , modelo f_θ , pérdida $L(\theta) = \frac{1}{N} \sum_i \ell(f_\theta(x_i), y_i)$.
- Entrenar = resolver $\min_\theta L(\theta)$ (en general no convexo).
- **Funciones de pérdida:**
 - Regresión: MSE $\ell(\hat{y}, y) = \frac{1}{2} \|\hat{y} - y\|^2$ (modelo gaussiano).
 - Clasificación: entropía cruzada $\ell = -\sum_i y_i \log \hat{y}_i$ con $\hat{y} = \text{softmax}(z)$ (modelo categórico).
- Elegir una pérdida es, en el fondo, elegir un modelo probabilístico de los datos.

1.6 Gradiente y descenso por gradiente

- $\nabla_\theta L \in \mathbb{R}^p$: vector de derivadas parciales, apunta a la dirección de máximo ascenso.
- Regla de actualización: $\theta \leftarrow \theta - \eta \nabla_\theta L(\theta)$.
- η grande $\rightarrow$ diverge; η chico $\rightarrow$ converge lento.
- **SGD**: gradiente estimado sobre un minibatch (insesgado).
- **Momentum**: acumula inercia. **Adam**: momentum + escala adaptativa por coordenada. Optimizador por defecto en la práctica.

1.7 Backpropagation como diferenciación automática

- **Regla de la cadena**: $\frac{\partial z}{\partial x} = \frac{\partial z}{\partial y} \cdot \frac{\partial y}{\partial x}$.
- Backprop = evaluar la cadena de la salida hacia la entrada, reutilizando resultados (producto de Jacobianos).
- **Modo reverso**: 1 escalar de salida (la pérdida), p parámetros $\Rightarrow$ gradiente completo en UNA pasada hacia atrás, costo $O(\text{forward})$. El modo directo costaría $O(p)$.
- **Grafo de cómputo**: nodos = valores intermedios; aristas = operaciones. Forward: evaluar en orden topológico. Backward: recorrer en orden inverso.

1.8 Notebook: Motor de autograd (micrograd)

El laboratorio construye un motor de diferenciación automática en modo reverso en ~ 100 líneas de Python:

1. **Value**: nodo del grafo. Guarda `data`, `grad` ($\partial L / \partial \text{self}$), `_prev` (padres), `_backward` (closure que propaga gradiente).
2. Operaciones (`_add_`, `_mul_`, `tanh`, `exp`): cada una crea un nodo, registra padres, y define su `_backward` con la derivada local.
3. **backward()**: orden topológico + barrido inverso disparando cada `_backward`.
4. **Gradient check**: verificar analítico $\approx$ numérico con diferencias finitas centradas. Regla de oro: *nunca confíes en un gradiente sin verificarlo*.
5. **Neuron** $\rightarrow$ **Layer** $\rightarrow$ **MLP**: todo se apoya en Value. El loop de entrenamiento (forward $\rightarrow$ loss $\rightarrow$ zero_grad $\rightarrow$ backward $\rightarrow$ update) es conceptualmente el mismo con que se entrena un LLM.

Punto clave: los gradientes se **acumulan** (`+=`), no se reemplazan. Por eso `zero_grad()` es imprescindible en cada paso.

Clase 2: Attention is All You Need — Transformers

2.1 El punto de partida

Paper: Vaswani et al., NeurIPS 2017. Introdujo el Transformer: arquitectura basada enteramente en atención, sin recurrencia ni convoluciones. Base de los modelos de lenguaje actuales.

2.2 Del texto a tokens

- Tres granularidades: carácter, subpalabra, palabra.
- V = tamaño del vocabulario (fijo). T = longitud de la secuencia (varía por texto).
- Trade-off: carácter = vocab chico pero secuencias largas; palabra = vocab enorme pero secuencias cortas. Subpalabra (BPE) busca el equilibrio.

2.3 El bigrama: el modelo más simple

- Predice el próximo carácter mirando únicamente el actual. Sin contexto, sin historia.
- `token_embedding_table = nn.Embedding(V, V)`: la fila i son los logits del próximo token dado el token i .
- De logits a probabilidades: `softmax(filac)`.
- Loss antes de entrenar: tabla random $\rightarrow$ predicciones picudas y equivocadas $\rightarrow$ loss $> \ln(V)$ (peor que el “ignorante honesto” uniforme).
- Entrenar un bigrama = contar pares. El gradiente en la fila: `grad = p - onehot(correcto)`.
- **Techo:** no conoce el contexto. Solo codifica “qué caracteres suelen seguir a c ” promediado sobre todo el corpus.

2.4 Promediar el pasado: darle memoria

- Representación de cada token = promedio de todos los tokens hasta él (bag of words del pasado). Ojo causal: solo hacia atrás.
- Tres versiones equivalentes:
 1. For-loop (correcto pero lento).
 2. Matriz triangular normalizada: `wei = tril/sum`, `xbow = wei @ x`.
 3. Softmax de afinidades: `zeros` $\rightarrow$ máscara $-\infty$ $\rightarrow$ softmax $\rightarrow$ `wei @ v`.
- **El molde que no cambia:** (1) armar `wei`, (2) máscara $-\infty$, (3) softmax por fila, (4) `salida = wei @ v`.
- **Problema del promedio:** descarta el **orden** y la **relevancia** (todos pesan igual).

2.5 Self-Attention

- Cambiamos UNA pieza del molde: w_i pasa a calcularse a partir de los tokens (ya no son ceros).
- Tres proyecciones lineales por token:
 - **Query** $q = x \cdot W_Q$: “qué estoy buscando”.
 - **Key** $k = x \cdot W_K$: “qué ofrezco para ser encontrado”.
 - **Value** $v = x \cdot W_V$: “qué apporto si me eligen”.
- Afinidad: $\text{wei}[i, j] = q_i \cdot k_j$ (producto interno).
- Fórmula de una cabeza:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + \text{máscara}\right) \cdot V$$

- Escalado $1/\sqrt{d_k}$: control de varianza. Sin él, el softmax se satura y el gradiente se apaga.
- **Positional encoding**: la atención es ciega al orden $\rightarrow$ entrada = embedding(token) + PE(pos).
- “Self”: Q, K, V salen de la misma secuencia. Si K, V vinieran de otra, sería cross-attention.

2.6 Multi-Head Attention

- Una cabeza = un solo par W_Q/W_K = una sola noción de “relevante”. Insuficiente: en lenguaje conviven varios tipos de relación (sintaxis, proximidad, coreferencia).
- Multi-head: H cabezas independientes, cada una con sus propias W_Q, W_K, W_V , en subespacios de tamaño C/H .
- Concatenar + proyección W_O : mezcla los bloques. Entra (B, T, C) , sale (B, T, C) .
- **Costo**: H cabezas de C/H tienen los mismos parámetros que 1 cabeza de C . Expresividad prácticamente gratis.

2.7 El bloque Transformer completo

- **FFN** (Feed-Forward Network): $\text{FFN}(x) = W_2 \cdot \phi(W_1 x + b_1) + b_2$. Cómputo por token (vertical), independiente por posición. Dimensión interna típica $4C$.
- Atención **comunica** tokens; FFN **procesa** cada token.
- **Conexiones residuales**: $x \leftarrow x + \text{Sublayer}(x)$. Camino directo para el gradiente $\rightarrow$ combate el desvanecimiento.
- **LayerNorm**: normaliza cada vector (media 0, varianza 1) sobre sus C componentes. Estabiliza activaciones.
- Bloque completo:

$$\begin{aligned}x &\leftarrow x + \text{MultiHead}(\text{LN}(x)) \\x &\leftarrow x + \text{FFN}(\text{LN}(x))\end{aligned}$$

- Se apilan N bloques. Capas bajas $\rightarrow$ estructura local; altas $\rightarrow$ relaciones abstractas. BERT: $N = 12$, GPT-3: $N = 96$.

2.8 Encoder vs Decoder

Encoder-only	Decoder-only	Encoder-decoder
BERT	GPT (lo que construimos)	T5, paper original
Comprensión	Generación	Secuencia a secuencia

- Encoder: atención bidireccional (sin máscara). Decoder: atención causal (con máscara).
- Cross-attention: Q del decoder, K y V del encoder.
- Se enmascara solo cuando ver un token sería hacer trampa respecto de lo que falta generar.

Clase 3: Detrás de la IA — Tokenizers

3.1 Por qué importa la tokenización

La tokenización está en el origen de muchos comportamientos anómalos de los LLMs. La unidad mínima es el **token**, no el carácter.

Comportamientos que explica:

- **Deletreo:** no deletrea ni cuenta letras dentro de un token.
- **Aritmética:** números segmentados de formas inconsistentes.
- **Otros idiomas:** más tokens por palabra fuera del inglés → peor rendimiento y mayor costo.
- **Espacios:** un espacio final puede alterar la tokenización.
- **Tokens raros** (SolidGoldMagikarp): token visto por el tokenizer pero casi no entrenado por el modelo → comportamiento errático.
- **Formato:** JSON vs YAML consume distinta cantidad de tokens.

3.2 De texto a bytes

- **Unicode:** asigna a cada carácter un **code point** (~ 150.000 definidos).
- Usar code points como vocabulario: $V \approx 150,000$, inestable, matrices enormes.
- **UTF-8:** reescribe cada code point como 1 a 4 **bytes** (0–255). Alfabeto fijo de 256, universal, sin fuera-de-vocabulario.
- ASCII: code point $\leq 127 \rightarrow 1$ byte (coincide con el número). Texto en inglés usa menos bytes.
- Límite del vocabulario de 256: T crece demasiado. Costo de atención $O(T^2)$.

3.3 Byte-Pair Encoding (BPE)

- Mecanismo: sobre los bytes, repetir: fusionar el par adyacente más frecuente en un token nuevo.
- Vocabulario inicial: 256 bytes (ids 0–255). Cada merge añade un token: id 256, 257, ...
- Tamaño final: $V = 256 + \text{número de merges}$ (hiperparámetro de diseño).
- **El entrenamiento no tiene gradiente:** es puro conteo de frecuencias sobre un corpus.
- Produce dos estructuras: **merges** (par $\rightarrow$ id nuevo, en orden) y **vocab** (id $\rightarrow$ bytes).

3.4 Encode y Decode

- **train**: descubre las reglas (por frecuencia). Se ejecuta una vez, offline.
- **encode(texto → ids)**: aplica los merges por **prioridad de merge** (orden en que se aprendieron), **no por frecuencia**. Texto → bytes → repetir: buscar el par presente cuyo merge se aprendió primero → fusionar.
- **decode(ids → texto)**: lookup en vocab. id → vocab[id] (bytes) → concatenar → decodificar UTF-8.
- **Round-trip exacto**: `decode(encode(texto)) == texto` para todo texto.
- Error común: encode por frecuencia en vez de por prioridad de merge.

3.5 Regex splitting (GPT-2/GPT-4)

- Sin restricciones, BPE fusiona a través de límites de palabra (`perro.`, `perro!` como tokens distintos).
- Solución: un patrón regex corta el texto en fragmentos (palabras, números, puntuación, espacios) **antes** de BPE. Los merges nunca cruzan fragmentos.
- GPT-4 vs GPT-2: contracciones case-insensitive, números en grupos de 1–3 dígitos, mejor manejo de espacios.
- Estos patrones se diseñan a mano, con sesgo hacia el inglés.

3.6 Tokens especiales

- Marcas de control fuera de BPE: `<|endoftext|>`, `<|im_start|>`, `<|im_end|>`, etc.
- Se agregan a mano con un id reservado, por encima de los bytes y merges.
- **encode** los separa antes de BPE. Riesgo de seguridad: hay que habilitarlos explícitamente (`allowed_special`).
- ChatML: `<|im_start|>user ... <|im_end|> <|im_start|>assistant.`

3.7 Tokenizers reales y algoritmos

Familia	Algoritmo	Librería
GPT (2/3/4)	BPE byte-level	tiktoken
BERT	WordPiece	BERT / HF tokenizers
T5	Unigram	SentencePiece
Llama 2 / Mistral	BPE (code points)	SentencePiece
Llama 3	BPE byte-level	estilo tiktoken

- **BPE**: bottom-up, fusiona pares más frecuentes.
- **WordPiece** (BERT): bottom-up, fusiona por máxima verosimilitud (no por frecuencia).
- **Unigram**: top-down, arranca con vocabulario grande y **poda** lo que menos aporta. Usa EM para estimar probabilidades, Viterbi para segmentar.
- **SentencePiece**: toolkit de Google. Corre BPE o Unigram sobre code points (no bytes). `byte_fallback` para OOV.

3.8 Notebook: Tokenización

Parte A — Tokenizer mínimo desde cero:

- Implementación de `get_stats` (contar pares) y `merge` (reemplazar par por id nuevo).
- 3 merges sobre `."aabdadaabac"` $\rightarrow$ `[258, 100, 258, 97, 99]`.
- Sobre texto real: 20 merges $\rightarrow$ compresión $\sim 1.57x$.
- Round-trip exacto verificado.
- Regex splitting con patrón de GPT-2: cada categoría (letras, números, puntuación) queda separada.

Parte B — Tokenizers reales:

- tiktoken: `cl100k` (GPT-4) vs `o200k` (GPT-4o). Vocabularios crecientes (`50k` $\rightarrow$ `100k` $\rightarrow$ `200k`).
- AutoTokenizer: GPT-2 (BPE, prefijo `G`), BERT (WordPiece, prefijo `##`), T5 (Unigram, símbolo `_`).
- Rarezas en vivo: `strawberry` $\rightarrow$ `[st, raw, berry]`; números partidos inconsistentemente; más tokens para japonés que inglés; JSON cuesta más tokens que YAML.

3.9 Notebook: “¿Qué es realmente la IA?”

Notebook introductorio usando Qwen3 8B con Ollama. Puntos clave:

- Un LLM **solo predice el próximo token**. Todo lo demás (memoria, personalidad, herramientas, búsqueda) es **software alrededor** del predictor.
- **raw=True**: apaga el chat template (el modelo recibe texto plano). **think=False**: apaga la cadena de razonamiento.
- La “conversación” es una ilusión: un único string con tokens especiales (`<|im_start|>`, `<|im_end|>`).
- **Temperatura**: controla el azar. $T = 0 \rightarrow$ determinístico; $T = 1 \rightarrow$ diverso.
- **Memoria**: cada llamada arranca de cero. Solución: reenviar TODO el historial.
- **Personalidad**: system prompt (texto al inicio del string).
- **Herramientas**: el modelo no calcula. Solución naive: router por keyword. Solución real: function calling.
- **Alucinación**: inventa con total seguridad cuando no sabe. Solución: RAG (inyectar contexto).
- **Alineamiento**: SFT (fine-tuning supervisado) + RLHF/preferencias. Convierte un simulador de texto en un asistente. Efecto colateral: adulación (sycophancy).
- Modelo base vs instruct: el base continúa texto; el instruct responde a instrucciones.

Clase 4: Embeddings

4.1 Parte 1: Word Embeddings

4.1.1. Los embeddings de un LLM se crean dentro del modelo

La tabla de embeddings (`nn.Embedding`) es un parámetro más, entrenado end-to-end con la misma loss de predecir el próximo token. Los modelos que vemos en esta clase son para entender el mecanismo.

4.1.2. Similitud coseno

$$\cos(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \|\mathbf{b}\|}$$

- $\cos = 1$: misma dirección (0°). $\cos = 0$: ortogonales (90°). $\cos = -1$: opuestos (180°).
- Distancia coseno: $\text{dist} = 1 - \cos$ (más chica = más cerca).
- Compara **dirección, no tamaño**. Un vector el doble de largo pero misma dirección da $\cos = 1$.
- Se usa coseno (no euclidiana) porque la magnitud correlaciona con frecuencia, y queremos factorizarla afuera.

4.1.3. One-hot: identidad, cero significado

- Vector de V ceros con un único 1. Gigante, sparse.
- $\cos(\text{gato}, \text{perro}) = 0 = \cos(\text{gato}, \text{avión})$: todo ortogonal a todo.
- **Identidad clave**: $\text{one_hot}(i) \cdot C = C[i]$. Un embedding es leer una fila de una tabla.

4.1.4. Hipótesis distribucional y conteo

- “Una palabra se conoce por la compañía que mantiene” (Firth, 1957).
- **Matriz de co-ocurrencia**: $M[i, j] = \text{veces que } j \text{ cae en la ventana de } i$.
- Problema del conteo crudo: lo domina la frecuencia. Palabras comunes co-ocurren con todo.
- **PPMI** (Positive Pointwise Mutual Information): $\text{PMI}(w, c) = \log \frac{P(w, c)}{P(w) \cdot P(c)}$, recortado a 0. Saca el efecto de frecuencia.
- **SVD** para comprimir la matriz. Cada palabra pasa de $|\text{contextos}|$ dimensiones a d .
- Emerge geometría (temas se agrupan), pero es ruidosa y no escala.

4.1.5. Word2Vec: aprender prediciendo

- **Truco central**: inventar una tarea de predicción descartable. La tarea es la excusa; los vectores son el premio.
- **Skip-gram**: dada una palabra central, predecir sus vecinas dentro de una ventana. Self-supervised.
- Score: $\text{score}(\text{centro}, \text{contexto}) = \mathbf{v}_{\text{centro}} \cdot \mathbf{u}_{\text{contexto}}$.

- **Cuello de botella:** softmax sobre todo V ($O(|V|)$ por par).
- **Negative sampling:** de “elegir entre V ” a “¿real o falso?”. $k + 1$ problemas binarios con sigmoide. $O(k)$ en vez de $O(V)$.
- Resultado: analogías como aritmética: rey – hombre + mujer $\approx$ reina.
- Descubrimiento (Levy & Goldberg 2014): skip-gram con negative sampling factoriza implícitamente una matriz de PMI.

4.1.6. GloVe: contar y después aprender

- Combina lo mejor de contar (global, eficiente) y predecir (patrones ricos).
- El objeto: el **cociente** $P(k|\text{ice})/P(k|\text{steam})$ discrimina; la probabilidad sola, no.
- Modelo: $\mathbf{v}_i \cdot \mathbf{u}_k + b_i + b_k = \log X_{ik}$. Regresión ponderada sobre la matriz de conteos.
- **No es una red:** profundidad cero, sin no linealidad, sin softmax. Modelo bilineal.
- Los tres (SVD+PPMI, skip-gram, GloVe) factorizan la misma matriz de asociación.
- Las analogías no son emergentes: son la restricción de diseño.

4.1.7. Techo: static embeddings y polisemia

Un vector fijo por palabra, ciego al contexto. “Bank” tiene dos sentidos pero un solo vector. La resolución: embeddings **contextuales** (self-attention / Transformer).

Arco completo: one-hot (identidad) $\rightarrow$ static (significado, ciego al contexto) $\rightarrow$ contextual (Transformer).

4.1.8. Sesgos

Los embeddings heredan los sesgos del corpus. Ejemplo: man : doctor :: woman : nurse. Medir y mitigar estos sesgos es parte de responsable AI.

4.2 Parte 2: Sentence Embeddings

4.2.1. El problema

Las aplicaciones operan sobre **textos**, no palabras sueltas. Necesitamos un vector por oración, de largo fijo, que capture el significado completo.

4.2.2. Baseline: promediar word vectors

- $\text{emb}(\text{oración}) = \frac{1}{n} \sum_{i=1}^n \mathbf{v}_i$. Centro de masa semántico.
- Funciona sorprendentemente bien como piso. Pero tiene tres agujeros:
 1. **Orden:** “the dog bit the man” y “the man bit the dog” $\rightarrow$ mismo vector (cos = 1).
 2. **Dilución:** oraciones largas promedian mucho relleno.
 3. **OOV:** palabras fuera de vocabulario se ignoran.

4.2.3. BERT: el encoder

- Encoder-only, bidireccional. Entrenado con masked language modeling.
- Produce un vector contextual por token. No genera texto.
- **BERT crudo no sirve para coseno**: nunca lo entrenaron para que cerca en significado $\Rightarrow$ cerca en coseno. Pooleado, rinde peor que promediar GloVe en STS.

4.2.4. SBERT (Sentence-BERT)

- SBERT = BERT + pooling + entrenamiento **contrastivo**.
- **Pooling**: de un vector por token a uno por oración. Mean pooling (más común) o token [CLS].
- **Entrenamiento contrastivo** (siamés): el mismo encoder embebe dos oraciones; la loss acerca las parecidas y aleja las distintas. Optimiza la geometría del coseno.
- Datos: pares etiquetados (paráfrasis, duplicados, NLI). Negativos gratis: in-batch negativos.
- Cierra los tres agujeros: self-attention lee el orden, pondera importancia, tokeniza en sub-words (sin OOV).
- Interfaz: `model.encode(una oración)` $\rightarrow$ un vector.

4.2.5. Embedders basados en decoder (GPT)

- Misma receta: vectores contextuales $\rightarrow$ poolear $\rightarrow$ fine-tuning contrastivo.
- Diferencia: se lee el **último** token (el único que vio toda la oración), no el [CLS].
- No se empieza de cero: arranca del GPT pre-entrenado. El pre-entrenamiento pone el **conocimiento**; el contrastivo pone la **métrica**.

4.2.6. Tres “embeddings” distintos

1. **De entrada**: vector fijo por token del vocabulario (lookup).
2. **Contextual por token**: salida de las capas con atención.
3. **Del texto** (el de RAG): UN solo vector para toda la query/documento.

4.2.7. Búsqueda semántica = la “R” de RAG

- Query + documentos $\rightarrow$ coseno $\rightarrow$ top- k . Eso es la recuperación de RAG.
- Match por **significado**, no por palabras. Una query puede no compartir palabras con el documento correcto y recuperarlo.
- **Invariante crítica**: query y documentos deben embeberse con el **mismo modelo**. El coseno solo tiene sentido dentro de un mismo espacio vectorial. Cambiar de embedder $\rightarrow$ re-indexar todo.
- Simétrico (comparar frases similares) vs asimétrico (query corta $\rightarrow$ pasaje largo).

4.2.8. Los tres modelos de un RAG

1. **Embedder** que indexa la base y embebe la query (mismo modelo).
2. **LLM generador** que escribe la respuesta (recibe texto, no vectores).
3. Se comunican por texto, no por vectores.

4.2.9. Qué embedder usar (2026)

- Benchmark: **MTEB**. Criterios: idioma, tamaño/latencia, licencia.
- APIs pagas: OpenAI, Cohere, Google (fuertes pero lock-in).
- Open source competitivo: Qwen3-Embedding, BGE-M3 (multilingüe).
- Livianos y locales: nomic-embed-text (137M, CPU), bge-m3.
- Regla: benchmarkear en tus propios datos.

4.2.10. Lo que falta para RAG en serio (Semana 8)

- **Chunking**: partir documentos largos en pedazos indexables.
- **Vector store**: Chroma, pgvector, Qdrant (en vez de lista de numpy).
- **Reranking**: segundo paso que afina el top- k .

4.3 Notebook: Word Embeddings (N1)

Recorrido ejecutable completo:

- §1: Coseno con números y plot (vectores 2D).
- §2: One-hot $\rightarrow$ lookup: `one_hot @ C == C[i]` verificado con `torch.allclose`.
- §3: Co-ocurrencia + PPMI + SVD desde cero sobre Reuters. Heatmaps de la matriz. Emerge geometría por temas (petróleo vs granos).
- §4: Word2Vec real con gensim sobre text8 (~ 17 M palabras). Vecinos de “king” y analogía king – man + woman $\approx$ queen.
- §5: GloVe pre-entrenado (400k palabras). Analogías limpias. Polisemia de “bank” (dominada por sentido financiero). Sesgos: doctor – man + woman $\approx$ nurse.
- §7: Skip-gram + negative sampling desde cero en PyTorch. Producto punto $\rightarrow$ logsigmoid. Con 50k tokens los vecinos son ruidosos: la geometría necesita escala.

4.4 Notebook: Sentence Embeddings (N2)

- §2: Baseline promedio. $\cos(\text{king}, \text{monarch}) = 0,929$ (alto), $\cos(\text{king}, \text{pizza}) = 0,759$ (bajo). Agujero del orden: dog/man $\rightarrow$ cos = 1,0.
- §3: SBERT (all-MiniLM-L6-v2 y multilingüe). Dog/man $\rightarrow$ cos = 0,98 (ya no idénticos).
- §4: Corpus de conceptos del curso. Heatmap de similitud: word2vec–negative sampling se iluminan juntos.
- §5: Búsqueda semántica a mano y con `util.semantic_search`. Query sin palabras en común recupera el documento correcto.

- §5: Invariante: $\cos(qA, qB) = 0.236$ con dos modelos distintos $\rightarrow$ espacios incompatibles.
- §6: RAG mínimo Ollama-nativo. Embedder: nomic-embed-text (768-dim). Generador: qwen3:8b.
Sin RAG inventa fechas; con RAG responde desde la base.

Lecturas recomendadas

Clase 1 Goodfellow, Bengio & Courville, *Deep Learning* (cap. 2, 4, 6.5). CS231n (Stanford). Repo micrograd (A. Karpathy).

Clase 2 Vaswani et al. (2017), *Attention Is All You Need*. Karpathy, nanoGPT. Brown et al. (2020), *Language Models are Few-Shot Learners* (GPT-3).

Clase 3 Sennrich et al. (2015), BPE en NLP. Radford et al. (2019), GPT-2 §2.2. Video: Karpathy, *Let's build the GPT Tokenizer*. Repo minbpe.

Clase 4 Lena Voita, *Word Embeddings*. CS224N (Stanford). Jay Alamar, *The Illustrated Word2vec*. Reimers & Gurevych (2019), *Sentence-BERT*. Leaderboard MTEB.